

实验五：鸿蒙多端应用开发

一、基本信息

项目	内容
学号	20232106445
姓名	张天慈
班级	软件 231
日期	2026.5.9
项目名称	智慧天气助手

二、实验目标

本实验旨在通过开发智慧天气助手应用，深入理解 HarmonyOS 的分布式架构和多端协同理念。实验要求学生熟练运用 ArkUI 声明式开发范式构建用户界面，掌握响应式布局设计和断点系统的实现方法。同时，学生需要了解设备传感器的工作原理和应用场景，理解分布式数据同步和跨设备协同的实现机制，培养模块化架构设计思维，实践低耦合、高内聚的软件工程原则。

三、实验环境

工具/资源	版本	用途
DevEco Studio	5.0.3.400	鸿蒙应用集成开发环境
HarmonyOS SDK	5.0.0.1	鸿蒙开发工具包
ArkTS	4.1.0	声明式编程语言
Nova15 模拟器	5.0	多设备模拟测试
TRAE AI 助手	2.5.0	辅助设计与开发

四、实验内容

阶段一：需求分析与设计

1.1 功能需求定义

基于智慧天气助手的定位，本应用包含以下核心功能模块。天气服务模块负责实时天气展示、多城市切换和生活指数功能，为用户提供全面的天气信息。环境感知模

块实现光线强度监测和自动主题切换功能，根据环境光照情况智能调整界面显示。设备姿态模块展示陀螺仪数据并提供方向指示功能，增强用户交互体验。数据同步模块实现用户设置的跨设备同步功能，确保用户偏好在不同设备间一致。个性化配置模块提供单位设置和主题选择功能，满足用户定制化需求。

1.2 用户体验设计

在交互设计方面，应用遵循直观操作原则，致力于减少用户操作步骤，提供一键式功能入口。同时，所有操作都配有明确的视觉反馈，确保用户清楚当前操作状态。应用具备智能感知能力，能够根据环境自动调整应用状态，提供更加智能的服务。响应迅速是用户体验的重要保障，确保页面切换和数据更新流畅无卡顿。

在视觉设计方面，应用采用蓝色作为主色调，配合渐变效果营造现代科技感。字体层级清晰分明，建立标题、正文、辅助文字的合理层级结构。间距规范统一，设置合理的边距和间距标准，确保界面整洁美观。动效设计注重平滑的过渡动画和状态切换效果，提升用户操作体验。

1.3 架构设计

采用分层架构设计，将系统分为表现层、服务层和数据层三个核心层次。表现层负责 UI 渲染与交互，包含 Index 页面和 Settings 页面等组件，直接面向用户提供操作界面。服务层封装业务逻辑，包括天气服务、传感器服务和同步服务等，采用单例模式确保全局状态一致性。数据层负责数据存储与管理，涵盖本地存储和分布式数据库，确保数据的持久化和跨设备同步。

阶段二：多端适配实践

2.1 响应式布局策略

采用基于组件层级的自适应布局策略，通过 ArkUI 提供的弹性布局能力实现多端适配。在弹性尺寸方面，使用百分比宽度和 `layoutWeight` 属性，确保组件在不同屏幕尺寸下按比例分配空间。在滚动容器方面，关键内容区域使用 `Scroll` 组件包裹，在小屏设备上支持垂直滚动浏览，避免内容溢出。在自适应间距方面，通过 `padding` 和 `margin` 属性设置合理间距，在不同设备上保持视觉舒适度。

2.2 自适应组件设计

核心自适应组件的设计充分考虑了多端适配需求。天气卡片组件根据屏幕宽度自动调整卡片大小和布局方式，在手机端采用单列布局，在平板端采用多列网格布局。城市选择器组件在小屏设备上使用下拉选择方式以节省空间，在大屏设备上使用横向滚动方式提升选择效率。传感器面板组件根据可用空间动态调整显示密度，确保信息展示的完整性。设置项组件采用响应式排列方式，支持单列和多列布局模式。

2.3 主题系统实现

应用实现了完整的日间/夜间双主题系统。日间主题采用浅色背景，呈现明亮的视觉效果，适合白天使用。夜间主题采用深色背景，减少视觉疲劳，适合暗光环境使用。自动模式根据光线传感器数据自动切换主题，无需用户手动干预。

主题切换机制支持多种方式。手动切换允许用户在设置页面自由选择主题模式。自动切换根据光线强度阈值自动进行主题切换，当光照强度超过 500lux 时切换为日间模式，低于 100lux 时切换为夜间模式。平滑过渡效果确保主题切换时界面动画流畅自然，使用 `animateTo` 实现 500ms 的平滑过渡，避免突兀的界面跳变。

阶段三：传感器集成探索

3.1 光线传感器应用

光线传感器通过光敏元件检测环境光照强度，返回值范围通常为 0-10000 lux。应用根据光照强度调整屏幕亮度和主题，为用户提供舒适的视觉体验。

在应用场景方面，光线传感器主要用于自动亮度调节功能，根据环境光自动调整屏幕亮度，提供舒适的观看体验。智能主题切换功能在暗光环境下自动切换到夜间模式，减少眼睛疲劳。节能控制功能在低光环境下降低屏幕功耗，延长设备续航时间。

在实现要点方面，需要设置合理的采样频率，默认 100ms 的采样间隔能够满足大多数场景需求。实现数据平滑处理算法，避免光照强度频繁波动导致的主题或亮度频繁切换。设置阈值区间和 hysteresis 机制，防止在阈值边界发生抖动现象。

3.2 陀螺仪传感器应用

陀螺仪传感器检测设备在三维空间中的旋转运动，输出 X、Y、Z 三个轴的角速度数据。这些数据可用于计算设备的姿态角度，实现多种交互功能。

在应用场景方面，陀螺仪传感器支持屏幕自动旋转功能，根据设备姿态自动调整屏幕方向。游戏控制功能将陀螺仪作为游戏控制器输入，提供沉浸式游戏体验。运动追踪功能记录用户运动轨迹，用于健身和健康应用。导航辅助功能结合 GPS 数据，提供更精准的定位和方向识别。

在实现要点方面，需要实现角度积分计算算法，将角速度数据转换为角度信息。添加噪声过滤算法剔除异常值，提高数据准确性。提供可视化角度展示功能，以图形方式直观显示设备姿态。

阶段四：分布式能力体验

4.1 分布式数据管理

本应用采用本地优先策略，确保在离线状态下应用仍能正常使用。在线时自动同步数据到云端，保证数据的时效性。支持多设备间的数据同步，用户在一个设备上的操作可以无缝同步到其他设备。

数据同步流程包含五个关键步骤。首先，用户修改设置后系统更新本地存储。其次，创建同步任务并加入待同步队列。然后，在网络可用时执行同步操作。接着，同步成功后更新同步状态。最后，通知所有订阅者并更新 UI 界面。

4.2 跨设备协同

在设备发现与连接方面，系统能够自动发现局域网内的鸿蒙设备，建立安全连接并交换设备信息。设备连接过程简单快捷，用户无需复杂配置即可实现设备协同。

在数据共享机制方面，支持用户偏好设置同步，确保用户在不同设备上获得一致的个性化体验。收藏数据可以在设备间共享，方便用户跨设备访问常用内容。状态信息实时同步，各设备间的应用状态保持一致。

五、实验结果截图

1、智慧天气助手应用界面，应用启动后自动定位用户位置并加载当地天气数据。



图 1 应用主界面

2、陀螺仪传感器数据展示界面，下方提供可视化方向指示器，直观展示设备当前姿态。下方为光线传感器数据展示界面，当前环境光照强度数值以及当前主题模式。



图 2 陀螺仪及光线传感器

3、天气界面包括当日气温、湿度、风速、风向、气压、能见度、紫外线等多种数据。



图 3 天气面板

4、设置页面整体截图，包含数据同步区域、显示设置区域、传感器设置区域和关于区域，各区域布局清晰，操作便捷。

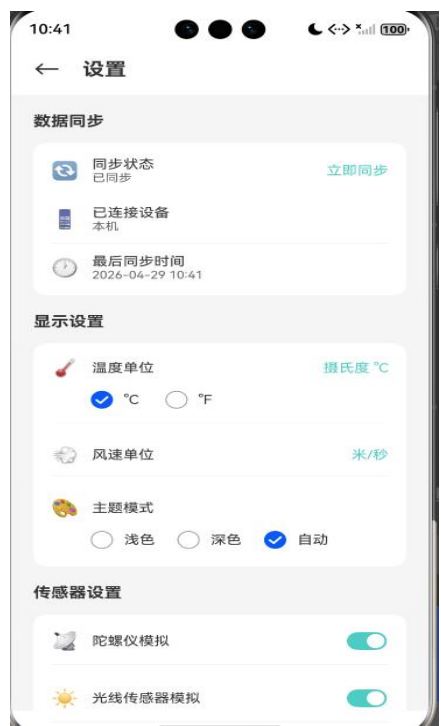


图 4 设置界面

六、技术分析

1、多端适配技术要点

在布局适配策略方面，本应用使用弹性布局容器实现自适应布局，通过 `layoutWeight` 分配空间比例，确保各组件根据屏幕尺寸自动调整。响应式断点切换布局模式，根据不同屏幕宽度切换到相应的布局方案。

在组件复用策略方面，抽取公共组件以减少代码重复，提高开发效率。使用 `@Builder` 装饰器复用 UI 片段，保持代码结构清晰。统一的样式管理确保应用整体风格一致。

2、传感器技术要点

在数据处理策略方面，采用数据平滑算法减少传感器数据的抖动，使用插值算法使数据变化更加自然。实现异常过滤机制，剔除异常值和噪声数据。设置合理的阈值控制，避免频繁触发导致的问题。

在资源管理策略方面，及时注册和注销传感器监听，避免内存泄漏。控制采样频率，平衡性能与功耗。后台运行时降低采样频率，减少电量消耗。

3、分布式技术要点

在数据一致性保障方面，使用版本号追踪数据变更，确保数据更新的顺序性。定义明确的冲突解决策略，处理多设备同时修改导致的冲突。本地缓存确保离线状态下的数据可用性。

在同步策略方面，定时同步定期检查并同步数据，保持数据的时效性。事件触发机制在数据变更时立即同步。批量同步减少网络请求次数，提高同步效率。

七、实验总结

通过本次实验，我深入掌握了 ArkUI 声明式开发范式，理解了多端适配的核心原理，熟悉了传感器数据的采集和处理方法，了解了分布式数据同步的实现机制。在技术能力方面，我学会了使用 `@State` 装饰器实现响应式 UI 更新，通过断点系统实现多端自适应，正确注册和注销传感器监听，以及将本地存储与在线同步相结合。在工程思维方面，我学会了模块化架构设计，理解了分层开发的优势，掌握了问题分析和解决的方法，培养了用户体验设计的意识。

我深刻体会到鸿蒙生态的独特优势。一次开发、多端部署的特性使得一套代码可以运行在多种设备上，大大提高了开发效率。分布式能力让设备间可以无缝协同，为用户带来连贯的使用体验。统一的开发体验确保了一致的 API 和开发工具，降低了学习成本。强大的生态支持提供了完善的开发工具链和丰富的文档资料。

我认为可以从以下几个方面进一步优化应用。增加更多传感器支持，如加速度计、磁力计等，拓展应用功能。实现更复杂的分布式协同场景，如多设备协同编辑等。优化性能和功耗管理，提升应用响应速度和续航表现。增加更多个性化功能，满足用户多样化的需求。